

Migrations (Göçler)

Kaynak: docs.djangoproject.com/en/6.0/topics/migrations/ — Türkçe Çeviri

Migrations (Göçler), modellerinizde yaptığınız değişiklikleri (alan ekleme, model silme vb.) veritabanı şemanıza yaymanın Django'nun yoludur. Büyük ölçüde otomatik olacak şekilde tasarlanmışlardır; ancak ne zaman migration oluşturmanız, ne zaman çalıştırmamız ve karşılaşılabileceğiniz genel sorunları bilmeniz gerekir.

1. Komutlar

Migrations ve Django'nun veritabanı şemasını işlemesiyle etkileşim için kullanacağınız birkaç komut bulunmaktadır:

- `migrate` — Migrationları uygulamaktan ve geri almaktan sorumludur.
- `makemigrations` — Modellerinizde yaptığınız değişikliklere göre yeni migrationlar oluşturmaktan sorumludur.
- `sqlmigrate` — Bir migration için SQL ifadelerini gösterir.
- `showmigrations` — Projenin migrationlarını ve durumlarını listeler.

Migrationları veritabanı şemanız için bir sürüm kontrol sistemi olarak düşünebilirsiniz. `makemigrations` model değişikliklerinizi bireysel migration dosyalarına paketlemekten sorumludur — bu, commit'lere benzer — ve `migrate` bunları veritabanınıza uygulamaktan sorumludur.

Her uygulama için migration dosyaları, o uygulamanın içindeki bir 'migrations' dizininde bulunur ve kod tabanının bir parçası olarak commit edilip dağıtılacak şekilde tasarlanmıştır. Bunları geliştirme makinenizde bir kez oluşturmalı, ardından aynı migrationları iş arkadaşlarınızın makinelerinde, staging makinelerinde ve sonunda üretim makinelerinizde çalıştırmalısınız.

Not: `MIGRATION_MODULES` ayarını değiştirerek her uygulama bazında migrationları içeren paketin adını geçersiz kılmak mümkündür.

Migrationlar aynı veri kümesinde aynı şekilde çalışır ve tutarlı sonuçlar üretir; yani geliştirme ve staging ortamında gördükleriniz, aynı koşullar altında üretimde tam olarak gerçekleşecek olanlarla aynıdır.

Django, modellerinizde veya alanlarınızda yapılan herhangi bir değişiklik için — veritabanını etkilemeyen seçenekler bile dahil olmak üzere — migration oluşturacaktır. Bunun nedeni, bir alanı doğru şekilde yeniden oluşturabilmesinin tek yolunun geçmişteki tüm değişikliklere sahip olmaktır; ve bu seçeneklere daha sonra bazı veri migrationlarında ihtiyaç duyabilirsiniz (örneğin, özel doğrulayıcılar ayarladıysanız).

2. Backend Desteği

Migrationlar, Django ile birlikte gelen tüm backend'lerde ve şema değişikliği desteği programlanmış herhangi bir üçüncü taraf backend'de desteklenmektedir (`SchemaEditor` sınıfı aracılığıyla yapılır). Ancak, şema migrasyonları açısından bazı veritabanları diğerlerine göre daha yeteneklidir.

2.1 PostgreSQL

PostgreSQL, şema desteği açısından buradaki tüm veritabanları arasında en yeteneklisidir.

2.2 MySQL

MySQL, şema değişiklik işlemleri etrafında transaction desteği eksikliği nedeniyle, bir migration uygulanamadığında tekrar denemek için değişiklikleri manuel olarak geri almanız gerekir (daha önceki bir noktaya geri dönemezsiniz).

MySQL 8.0, DDL işlemleri için önemli performans iyileştirmeleri sunarak bunları daha verimli hale getirmiş ve tam tablo yeniden oluşturma ihtiyacını azaltmıştır. Ancak, kilitleme veya kesintilerin tam yokluğunu garanti edemez. Kilitlerin hâlâ gerekli olduğu durumlarda, bu işlemlerin süresi ilgili satır sayısı ile orantılı olacaktır.

Son olarak, MySQL'in bir indeksin kapsadığı tüm sütunların birleşik boyutu üzerinde görece küçük bir sınırı vardır. Bu, diğer backend'lerde mümkün olan indekslerin MySQL altında oluşturulamayacağı anlamına gelir.

2.3 SQLite

SQLite'in çok az yerleşik şema değişiklik desteği vardır, bu nedenle Django bunu şunları yaparak taklit etmeye çalışır:

- Yeni şema ile yeni bir tablo oluşturma
- Verileri kopyalama
- Eski tabloyu silme
- Yeni tabloyu orijinal adla yeniden adlandırma

Bu süreç genellikle iyi çalışır, ancak yavaş ve zaman zaman hatalı olabilir. Risklerin ve sınırlamaların çok farkında olmadıkça, üretim ortamında SQLite'ı çalıştırıp migrate etmeniz önerilmez.

3. İş Akışı (Workflow)

Django sizin için migration oluşturabilir. Modellerinizde değişiklik yapın — örneğin bir alan ekleyin ve bir model kaldırın — ardından `makemigrations` komutunu çalıştırın:

```
$ python manage.py makemigrations
Migrations for 'books':
books/migrations/0003_auto.py:
~ Alter field author on book
```

Modelleriniz taranacak ve şu anda migration dosyalarınızda bulunan sürümlerle karşılaştırılacak, ardından yeni bir migration seti yazılacaktır. `makemigrations`'in neyi değiştirdiğinizi düşündüğünü görmek için çıktıyı okuyun — bu araç kusursuz değildir.

Yeni migration dosyalarınızı edindikten sonra, beklendiğinden emin olmak için bunları veritabanınıza uygulamalısınız:

```
$ python manage.py migrate
Operations to perform:
Apply all migrations: books
Running migrations:
Applying books.0003_auto... OK
```

Migration uygulandıktan sonra, migration ve model değişikliğini sürüm kontrol sisteminize tek bir commit olarak işleyin — bu sayede diğer geliştiriciler kodu kontrol ettiğinde, hem modellerinizdeki değişiklikleri hem de eşlik eden migrationı aynı anda alacaklardır.

Oluşturulan bir ad yerine migration'a anlamlı bir ad vermek isterseniz, `makemigrations --name` seçeneğini kullanabilirsiniz:

```
$ python manage.py makemigrations --name changed_my_model your_app_label
```

3.1 Sürüm Kontrolü

Migrationlar sürüm kontrolünde saklandığından, zaman zaman siz ve başka bir geliştirici aynı anda aynı uygulamaya bir migration işlemiş olursunuz; bu da aynı numaraya sahip iki migration ile sonuçlanır.

Endişelenmeyin — numaralar yalnızca geliştiricilerin referansı için oradadır, Django yalnızca her migration'ın farklı bir ada sahip olmasını öner. Bu olduğunda, Django sizi uyarır ve size birkaç seçenek sunar. Yeterince güvenli olduğunu düşünüyorsa, iki migrationı sizin için otomatik olarak doğrusallaştırmayı teklif edecektir.

4. Transaction'lar (İşlemler)

DDL transaction'larını destekleyen veritabanlarında (SQLite ve PostgreSQL), tüm migration işlemleri varsayılan olarak tek bir transaction içinde çalıştırılır. Bunun aksine, bir veritabanı DDL transaction'larını desteklemiyorsa (örneğin MySQL, Oracle), tüm işlemler transaction olmadan çalıştırılır.

Bir migration'ın transaction içinde çalışmasını, `atomic` özelliğini `False` olarak ayarlayarak önleyebilirsiniz. Örneğin:

```
from django.db import migrations

class Migration(migrations.Migration):
    atomic = False
```

Ayrıca, `atomic()` kullanarak veya `RunPython`'a `atomic=True` geçirerek migration'ın bazı bölümlerini bir transaction içinde yürütmek de mümkündür.

5. Bağımlılıklar (Dependencies)

Migrationlar uygulama bazlı olsa da, modellerinizin ima ettiği tablolar ve ilişkiler bir anda bir uygulama için oluşturulamayacak kadar karmaşıktır. Çalıştırılması başka bir şeye bağımlı bir migration yaptığınızda — örneğin, `books` uygulamanızdaki `authors` uygulamanıza bir `ForeignKey` eklediğinizde — ortaya çıkan migration, `authors` uygulamasındaki bir migration'a bağımlılık

içerecektir.

Bu, migrationları çalıştırdığınızda, `authors` migration'ının önce çalıştırılıp `ForeignKey`'in referans verdiği tabloyu oluşturacağı ve ardından `ForeignKey` sütununu oluşturan migration'ın çalışacağı anlamına gelir.

Önemli: Migrationsız uygulamalar, migration'lara sahip uygulamalarla ilişkilere (`ForeignKey`, `ManyToManyField` vb.) sahip olmamalıdır. Bazen çalışabilir, ancak desteklenmez.

5.1 Değiştirilebilir Bağımlılıklar (Swappable Dependencies)

`swappable_dependency(value)` fonksiyonu, migration'larda değiştirilebilir modelin uygulamasındaki migration'lara — şu an itibarıyla bu uygulamanın ilk migration'una — 'değiştirilebilir' bağımlılıklar bildirmek için kullanılır. `value` argümanı, uygulama etiketi ve model adını açıklayan `"."` biçimindeki bir string'dir; örneğin `"myapp.MyModel"`.

`swappable_dependency()` kullanarak, Django'nun kimlik doğrulama sisteminde `settings.AUTH_USER_MODEL` (varsayılan olarak `"auth.User"`) gibi özelleştirme veya değiştirmeye tabi modellere referans vermek için kullanılır.

6. Migration Dosyaları

Migrationlar, burada 'migration dosyaları' olarak adlandırılan disk üzerindeki bir formatta saklanır. Bu dosyalar aslında, bildirimsel bir tarzda yazılmış, üzerinde anlaşılmış bir nesne düzenine sahip normal Python dosyalarıdır. Temel bir migration dosyası şöyle görünür:

```
from django.db import migrations, models

class Migration(migrations.Migration):
    dependencies = [("migrations", "0001_initial")]

    operations = [
        migrations.DeleteModel("Tribble"),
        migrations.AddField("Author", "rating", models.IntegerField(default=0)),
    ]
```

Django bir migration dosyasını yüklediğinde aradığı şey, `Migration` adlı `django.db.migrations.Migration`'in bir alt sınıfıdır. Ardından bu nesneyi dört özellik açısından inceler; bunların yalnızca ikisi çoğu zaman kullanılır:

- `dependencies` — Bu migration'ın bağımlı olduğu migration'ların listesi.
- `operations` — Bu migration'ın ne yapacağını tanımlayan `Operation` sınıflarının listesi.

İşlemler anahtardır; Django'ya hangi şema değişikliklerinin yapılması gerektiğini söyleyen bildirimsel talimatlardır. Django bunları tarar ve tüm uygulamalardaki tüm şema değişikliklerinin bellekte bir temsilini oluşturur; ardından şema değişikliklerini yapan SQL'i üretmek için bunu kullanır.

Migration dosyalarını nadiren veya hiç elle düzenlemeniz gerekmez, ancak gerekirse bunları manuel olarak yazmak tamamen mümkündür.

6.1 Özel Alanlar (Custom Fields)

Bir `TypeError` hatası almadan, hâlihazırda migrate edilmiş bir özel alandaki konumsal argüman sayısını değiştiremezsiniz. Yeni bir argümana ihtiyaç duyarsanız lütfen bir anahtar kelime argümanı oluşturun ve kurucuya `assert arg_name in kwargs` gibi bir şey ekleyin.

6.2 Model Manager'ları

Manager'ları isteğe bağlı olarak migration'lara serileştirebilir ve bunları `RunPython` işlemlerinde kullanılabilir hale getirebilirsiniz. Bu, manager sınıfında bir `use_in_migrations` özelliği tanımlanarak yapılır:

```
class MyManager(models.Manager):
    use_in_migrations = True

class MyModel(models.Model):
    objects = MyManager()
```

6.3 İlk Migration'lar (Initial Migrations)

Bir uygulama için 'ilk migration'lar', uygulamanın tablolarının ilk sürümünü oluşturan migration'lardır. İlk migration'lar, migration sınıfındaki bir `initial = True` sınıf özelliğiyle işaretlenir.

`migrate --fake-initial` seçeneği kullanıldığında, bu ilk migration'lar özel olarak ele alınır. Bir veya daha fazla tablo oluşturan (`CreateModel` işlemi) bir ilk migration için Django, bu tabloların hepsinin veritabanında zaten mevcut olup olmadığını kontrol eder.

6.4 Geçmiş Tutarlılığı (History Consistency)

İki geliştirme dalı birleştirildiğinde migration'ları manuel olarak doğrusallaştırmanız gerekebilir. Migration bağımlılıklarını düzenlerken tutarsız bir geçmiş durumu yaratabilirsiniz. Bu, bağımlılıkların yanlış olduğunu gösterir; bu nedenle Django, düzeltilene kadar migration'ları çalıştırmayı veya yeni migration'lar oluşturmayı reddedecektir.

7. Uygulamalara Migration Ekleme

Uygulamanız zaten modellere ve veritabanı tablolarına sahipse ve henüz migration'lara sahip değilse, onu migration'ları kullanacak şekilde dönüştürmek için şunu çalıştırmanız gerekir:

```
$ python manage.py makemigrations your_app_label
```

Bu, uygulamanız için yeni bir ilk migration oluşturacaktır. Şimdi, `python manage.py migrate --fake-initial` komutunu çalıştırın; Django bir ilk migration'a sahip olduğunuzu ve oluşturmak istediği tabloların zaten mevcut olduğunu tespit edecek ve migration'u hâlihazırda uygulanmış olarak işaretleyecektir.

Not: Bunun yalnızca iki koşul altında işleyeceğini unutmayın: (1) Tablolarınızı oluşturduğunuzdan bu yana modellerinizi değiştirmediniz. (2) Veritabanınızı manuel olarak düzenlemedi.

8. Migration'ları Geri Alma (Reversing)

Migration'lar, önceki migration'ın numarası geçilerek `migrate` komutuyla geri alınabilir. Örneğin, `books.0003` migration'ını geri almak için:

```
$ python manage.py migrate books 0002
Operations to perform:
Target specific migration: 0002_auto, from books
Running migrations:
Rendering model states... DONE
Unapplying books.0003_auto... OK
```

Bir uygulama için uygulanan tüm migration'ları geri almak isterseniz `zero` adını kullanın:

```
$ python manage.py migrate books zero
Operations to perform:
Unapply all migrations: books
Running migrations:
Rendering model states... DONE
Unapplying books.0002_auto... OK
Unapplying books.0001_initial... OK
```

Bir migration, geri alınamaz işlemler içeriyorsa geri alınamaz. Böyle migration'ları geri almaya çalışmak `IrreversibleError` hatası verir.

9. Geçmiş Modeller (Historical Models)

Migration'ları çalıştırdığınızda, Django migration dosyalarında saklanan modellerinizin tarihi sürümlerini kullanır. `RunPython` işlemini kullanan Python kodu yazıyorsanız veya veritabanı yönlendiricinizde `allow_migrate` metodlarınız varsa, bunları doğrudan içeri aktarmak yerine bu tarihi model sürümlerini kullanmanız **gerekir**.

Uyarı: Tarihi modeller yerine modelleri doğrudan içeri aktarırsanız, migration'larınız başlangıçta çalışabilir; ancak eski migration'ları yeniden çalıştırmaya çalıştığınızda (genellikle yeni bir kurulum yapıp veritabanını kurmak için tüm migration'ları çalıştırdığınızda) gelecekte başarısız olacaktır.

Uyarı: Migrationlarda nesnelere erişiminizde özel `save()` metodlarının **ÇAĞIRILMAYACAĞI** ve herhangi bir özel yapıcı veya örnek metodlarına sahip **OLMAYACAĞINIZ** anlamına gelir. Buna göre plan yapın!

Geçmiş model sorunlarını gidermek için `squash migrations` yapabilir veya referansları doğrudan migration dosyalarına kopyalayabilirsiniz.

10. Model Alanları Kaldırılırken Dikkat Edilecekler

Özel model alanlarını projenizden veya üçüncü taraf uygulamadan kaldırmak, eski migration'larda referans alındıkları takdirde soruna neden olur. Bu durumla başa çıkmak için Django, sistem kontrol çerçevesini kullanan model alanı kullanımdan kaldırma işlemine yardımcı olmak için bazı model alanı özellikleri sağlar.

Model alanınıza `system_check_deprecated_details` özelliğini ekleyin:

```
class IPAddressField(Field):
    system_check_deprecated_details = {
        "msg": (
            "IPAddressField has been deprecated. Support for it (except "
```

```
"in historical migrations) will be removed in Django 1.9."
),
"hint": "Use GenericIPAddressField instead.",
"id": "fields.W900",
}
```

Seçtiğiniz bir kullanımdan kaldırma sürecinden sonra, `system_check_deprecated_details` özelliğini `system_check_removed_details` olarak değiştirin ve sözlüğü güncelleyin.

```
class IPAddressField(Field):
    system_check_removed_details = {
        "msg": (
            "IPAddressField has been removed except for support in "
            "historical migrations."
        ),
        "hint": "Use GenericIPAddressField instead.",
        "id": "fields.E900",
    }
```

11. Veri Migration'ları (Data Migrations)

Şema değiştirmenin yanı sıra, migration'ları veritabanındaki verileri de değiştirmek için kullanabilirsiniz. Veri değiştiren migration'lara genellikle 'veri migration'ları' denir; bunlar en iyi şema migration'larının yanına ayrı migration'lar olarak yazılır. Django veri migration'larını sizin için otomatik olarak oluşturamaz, ancak ana işlem `RunPython`'dır.

Başlamak için boş bir migration dosyası oluşturun:

```
python manage.py makemigrations --empty yourappname
```

Dosya şuna benzer görünmelidir:

```
# Generated by Django A.B on YYYY-MM-DD HH:MM
from django.db import migrations

class Migration(migrations.Migration):
    dependencies = [
        ("yourappname", "0001_initial"),
    ]

    operations = []
```

Şimdi yapmanız gereken tek şey yeni bir fonksiyon oluşturmak ve `RunPython`'ın bunu kullanmasını sağlamaktır. Yeni `name` alanını `first_name` ve `last_name` değerlerinin birleşimiyle dolduran bir migration yazalım:

```
from django.db import migrations

def combine_names(apps, schema_editor):
    # Person modelini doğrudan import edemeyiz çünkü
    # bu migration'ın beklediğinden daha yeni bir sürüm olabilir.
    # Tarihi sürümü kullanıyoruz.
```

```
Person = apps.get_model("yourappname", "Person")
for person in Person.objects.all():
    person.name = f"{person.first_name} {person.last_name}"
    person.save()
```

```
class Migration(migrations.Migration):
    dependencies = [
        ("yourappname", "0001_initial"),
    ]

    operations = [
        migrations.RunPython(combine_names),
    ]
```

Bu yapıldıktan sonra, her zamanki gibi `python manage.py migrate` komutunu çalıştırabilirsiniz ve veri migration'u diğer migration'ların yanında yerinde çalıştırılacaktır.

Geri migration yaparken çalıştırılmasını istediğiniz mantığı çalıştırmak için `RunPython`'a ikinci bir callable geçebilirsiniz. Bu callable atlanırsa, geri migrate etmek bir istisna oluşturacaktır.

11.1 Diğer Uygulamalardan Modellere Erişme

Migration'un bulunduğu uygulamanın dışındaki uygulamalardaki modelleri kullanan bir `RunPython` fonksiyonu yazarken, migration'un `dependencies` özelliği ilgili her uygulamanın en son migration'unu içermelidir; aksi takdirde `LookupError: No installed app with label` gibi bir hata alabilirsiniz.

```
class Migration(migrations.Migration):
    dependencies = [
        ("app1", "0001_initial"),
        # app2'deki modelleri kullanmak için bağımlılık
        ("app2", "0004foobar"),
    ]

    operations = [
        migrations.RunPython(move_m1),
    ]
```

12. Migration'ları Sıkıştırma (Squashing)

Migration'ları özgürce oluşturmanız ve kaç tane olduğunuz konusunda endişelenmeniz önerilir; migration kodu, birçok yavaşlama olmadan aynı anda yüzlerle başa çıkacak şekilde optimize edilmiştir. Ancak eninde sonunda yüz kadar migration'dan birkaç tanesine geri dönmek isteyeceksiniz ve işte sıkıştırma devreye giriyor.

Sıkıştırma, hâlâ aynı değişiklikleri temsil eden bir (veya bazen birkaç) migration'a indirgemektir. Django bunu; tüm mevcut migration'larını alarak, `Operation`'larını çıkarıp hepsini sıraya koyarak ve ardından listeyi azaltmaya çalışmak için bunların üzerinde bir optimizör çalıştırarak yapar. Örneğin, `CreateModel` ve `DeleteModel`'in birbirini iptal ettiğini ve `AddField`'in `CreateModel` içine yuvarlanabileceğini bilir.


```
$ ./manage.py squashmigrations myapp 0004
Will squash the following migrations:
- 0001_initial
- 0002_some_change
- 0003_another_change
- 0004_undo_something
Do you wish to proceed? [y/N] y
Optimizing...
Optimized from 12 operations to 7 operations.
Created new squashed migration /home/.../0001_squashed_0004_undo_something.py
```

Otomatik olarak oluşturulan bir ad yerine sıkıştırılmış migration'ın adını kendiniz belirlemek isterseniz `squashmigrations --squashed-name` seçeneğini kullanın.

Sıkıştırılmış migration'u normal bir migration'a geçirmek için:

- Yerini aldığı tüm migration dosyalarını silin.
- Silinen migration'lara bağımlı olan tüm migration'ları, bunun yerine sıkıştırılmış migration'a bağımlı olacak şekilde güncelleyin.
- Sıkıştırılmış migration'un `Migration` sınıfındaki `replaces` özelliğini kaldırın.

Not: Django 6.0'da sıkıştırılmış migration'ların kendisini tekrar sıkıştırma desteği eklendi.

13. Değerleri Serileştirme (Serializing Values)

Migration'lar, modellerinizin eski tanımlarını içeren Python dosyalarıdır. Bu nedenle Django, modellerinizin mevcut durumunu alabilmeli ve bunları bir dosyaya serileştirmelidir.

Django aşağıdakileri serileştirebilir:

- `int`, `float`, `bool`, `str`, `bytes`, `None`, `NoneType`
- `list`, `set`, `tuple`, `dict`, `range`
- `datetime.date`, `datetime.time` ve `datetime.datetime` örnekleri (saat dilimi farkındakiler dahil)
- `zoneinfo.ZoneInfo` örnekleri (Django 6.0'da eklendi)
- `decimal.Decimal` örnekleri
- `enum.Enum` ve `enum.Flag` örnekleri
- `uuid.UUID` örnekleri
- Serileştirilebilir değerlere sahip `functools.partial()` ve `functools.partialmethod` örnekleri
- `pathlib`'den saf ve somut yol nesneleri
- `os.PathLike` örnekleri
- Serileştirilebilir bir değeri saran `LazyObject` örnekleri
- `TextChoices` veya `IntegerChoices` gibi enumeration türü örnekleri
- Herhangi bir Django alanı
- Herhangi bir fonksiyon veya metot referansı (modülün üst düzey kapsamında olmalıdır)
- Özel bir `deconstruct()` metoduna sahip herhangi bir şey

Django aşağıdakileri serileştiremez:

- İç içe sınıflar

- Rastgele sınıf örnekleri (örneğin `MyClass(4.3, 5.7)`)
- Lambda'lar

13.1 Özel Serileştiriciler (Custom Serializers)

Özel bir serileştirici yazarak diğer türleri serileştirebilirsiniz. Örneğin, Django varsayılan olarak `Decimal` serileştirmiyorsa, bunu yapabilirsiniz:

```
from decimal import Decimal
from django.db.migrations.serializer import BaseSerializer
from django.db.migrations.writer import MigrationWriter

class DecimalSerializer(BaseSerializer):
    def serialize(self):
        return repr(self.value), {"from decimal import Decimal"}

MigrationWriter.register_serializer(Decimal, DecimalSerializer)
```

13.2 deconstruct() Metodu Ekleme

Sınıfa bir `deconstruct()` metodu ekleyerek Django'nun kendi özel sınıf örneklerini serileştirmesine izin verebilirsiniz. Bu metot argüman almaz ve üç şeyden oluşan bir demet döndürmelidir: (`path`, `args`, `kwargs`):

- `path` — Sınıfın Python yolu olmalıdır (örneğin `myapp.custom_things.MyClass`).
- `args` — Sınıfın `__init__` metoduna geçilecek konumsal argümanların listesi.
- `kwargs` — Sınıfın `__init__` metoduna geçilecek anahtar kelime argümanlarının sözlüğü.

```
from django.utils.deconstruct import deconstructible

@deconstructible
class MyCustomClass:
    def __init__(self, foo=1):
        self.foo = foo
    ...

    def __eq__(self, other):
        return self.foo == other.foo
```

14. Birden Fazla Django Sürümünü Destekleme

Modellere sahip bir üçüncü taraf uygulamanın geliştiricisiyseniz, birden fazla Django sürümünü destekleyen migration'ları göndermeniz gerekebilir. Bu durumda, `makemigrations` komutunu desteklemek istediğiniz **en düşük Django sürümüyle** çalıştırmalısınız.

Migration sistemi, Django'nun geri kalan kısımlarıyla aynı politikaya göre geriye dönük uyumluluğu koruyacaktır; bu nedenle Django X.Y'de oluşturulan migration dosyaları Django X.Y+1'de değiştirilmeksizin çalışmalıdır. Migration sistemi ileriye dönük uyumluluğu garanti etmez; yeni özellikler eklenebilir ve Django'nun yeni sürümlerinde oluşturulan migration dosyaları eski sürümlerinde çalışmayabilir.

